

Parth Patel
CSCI 4104

11/9/23
Problem Set 4

26.1-6 Professor Karan measures her deterministic task-parallel algorithm on 4, 10, and 64 processors of an ideal parallel computer using a greedy scheduler. She claims that the three runs yielded $T_4 = 80$ seconds, $T_{10} = 42$ seconds, and $T_{64} = 10$ seconds. Argue that the professor is either lying or incompetent. (Hint: Use the work law (26.2), the span law (26.3), and inequality (26.5) from Exercise 26.1-4)

Inequality from Exercise 26.1-4: $T_P \leq \frac{T_1 - T_\infty}{P} + T_\infty$

By the work law: $T_4 = 80 \geq \frac{T_1}{4}$, so then, $320 \geq T_1$

By the span law: $T_\infty \leq T_{64} = 10$, which means $T_\infty \leq 10$

By the inequality: Let $P = 10$, so that:

$$T_{10} = 42 \leq \frac{T_1 - T_\infty}{10} + T_\infty$$

$$= 42 \leq \frac{T_1 - T_\infty + 10T_\infty}{10}$$

$$= 42 \leq \frac{T_1 + 9T_\infty}{10}$$

$$= 420 \leq T_1 + 9T_\infty$$

$$\text{Since } T_1 \leq 320, \text{ then } 420 \leq 320 + 9T_\infty$$

$$\text{Then, } 100 \leq 9T_\infty$$

$$\frac{100}{9} \leq T_\infty, \text{ and } 10 < \frac{100}{9} = 11.\overline{11}$$

$$\text{So, } 10 < T_\infty$$

However, this is a contradiction, since by the span law above, we know that $T_\infty \leq 10$, not $T_\infty > 10$.

Thus, Professor Karan is either lying or incompetent.

26.1-8 Analyze the work, span, and parallelism of the procedure P-TRANSPOSE, which transposes an $n \times n$ matrix A in place.

P-TRANSPOSE(A, n)

parallel for $j=2$ to n

parallel for $i=1$ to $j-1$

exchange a_{ij} with a_{ji}

Work: $n \times n \times 1 = n^2 = O(n^2)$

Span: $\lg n + \lg n + 1 = 2\lg n + 1 = O(\lg n)$

Parallelism: $\frac{n^2}{\lg n} = O(\frac{n^2}{\lg n})$

26.1-9 Suppose that instead of a parallel for loop in line 2, the P-TRANSPOSE procedure in Exercise 26.1-8 had an ordinary for loop. Analyze the work, span, and parallelism of the resulting algorithm.

P-TRANSPOSE(A, n)

parallel for $j=2$ to n

for $i=1$ to $j-1$

exchange a_{ij} with a_{ji}

Work: $n \times n \times 1 = n^2 = O(n^2)$

Span: $\lg n + (n-1)(1) = \lg n + (n-1) = O(n)$

Parallelism: $\frac{n^2}{n} = n = O(n)$

26.1-10 For what number of processors do the two versions of the chess program run equally fast, assuming that

$T_p = T_1/p + T_{\infty}$?

Given: $T_1 = 2048$ seconds, $T_{\infty} = 1$ second, $T'_1 = 1024$ seconds,

$T'_{\infty} = 8$ seconds

Then $T_p = T'_p \rightarrow \frac{T_1}{p} + T_{\infty} = \frac{T'_1}{p} + T'_{\infty} \rightarrow \frac{2048}{p} + 1 = \frac{1024}{p} + 8$

$\rightarrow \frac{2048}{p} = \frac{1024}{p} + 7 \rightarrow 2048 = 1024 + 7p \rightarrow 1024 = 7p \rightarrow p \approx 146.286$

Since we can't have a fraction of a processor, then the solution here will either have to be $P = 146$ processors, or $P = 147$ processors

26.2-5 Give pseudocode for an efficient parallel version of the Floyd-Warshall algorithm (see Section 23.2), which computes shortest paths between all pairs of vertices in an edge-weighted graph. Analyze your algorithm.

Floyd-Warshall (W : an $n \times n$ weighted adjacency matrix, n : number of vertices)

Let D be an $n \times n$ weighted adjacency matrix that is a copy of W

Let P be an $n \times n$ matrix

parallel for $i = 1$ to n

parallel for $j = 1$ to n

if $i \neq j$ and $W[i][j] < \infty$

$P[i][j] = i$

else

$P[i][j] = \text{NIL}$

// Note that I am making this outermost for-loop a

// regular loop, since each iteration of "k" will depend

// on the version of "D" created by the previous iteration of "k".

// So, this outermost loop cannot run parallel

for $k = 1$ to n

parallel for $i = 1$ to n

parallel for $j = 1$ to n

$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$

$P[i][j] = P[k][j]$

return D, P

Analysis:

Work: $(n \times n) + (1) + (n \times n \times 1) + (n \times n \times n \times 1) = n^3 + 2n^2 + 1 = O(n^3)$

Span: $(n \times n) + (1) + (\lg(n) + \lg(n) + 1) + n(\lg(n) + \lg(n) + 1) =$
 $= n^2 + 2 + 2\lg(n) + 2n \cdot \lg(n) + n = O(n \cdot \lg(n))$

Parallelism: $\frac{n^3}{n \cdot \lg(n)} = \frac{n^2}{\lg(n)} = O(\frac{n^2}{\lg(n)})$

Speedup: $T_p = (n \times n) + (1) + (\frac{n^2}{P} + \lg(P) + \lg(P) + 1) +$
 $n(\frac{n^2}{P} + \lg(P) + \lg(P) + 1) =$

$T_p = n^2 + 2 + \frac{n^2}{P} + 2\lg(P) + \frac{n^3}{P} + 2n \cdot \lg(P) + n = O(\frac{n^3}{P} + 2n \cdot \lg(P))$

$\frac{T_1}{T_p} = O(\frac{n^3}{\frac{n^3}{P} + 2n \cdot \lg(P)})$